

Activity Dossier

Lab 5 — Deep Learning on Trial: Can You Certify a Neural Network?

AI & Data Science with GenAI (Rail) · Deep Learning — your DL mini case and safety case

Student package — everything you need to do this activity

Field	Detail
Module	Deep Learning — Neural Networks
Builds on	Lab 2 (the same dataset, the same split, the one-line baseline) and Lab 4 (never tune on the test set)
Duration	~2 hours (about 80 minutes on the notebook and Challenge, the rest starting the safety case)
You need	A browser and a free Google account. Nothing to install.
Data	Real MetroPT-3 APU sensors, 144,437 minutes (Feb – Jun 2020) — the same file as Lab 2
Hand in	Your completed notebook + a written Safety Case

1. The situation

A supplier has approached the depot with a product: an APU Health Monitor built on a deep neural network. Their pitch is short and, on the face of it, compelling:

The supplier's claim. "Unlike your current rule-based alarms, our deep model needs no failure examples to train. It learns what a healthy Air Production Unit looks like and flags any abnormal behaviour — including fault types you have never seen before. Fleet-wide, always on."

That claim is attractive, and the technology behind it is real. Your job today is not to admire it and not to dismiss it. You will build the supplier's model yourself, on data you control, measure it honestly, and then decide — with evidence — whether you would let it anywhere near a safety-related duty.

You are working with the same dataset as Lab 2: real sensor readings from a metro train's compressor, with two genuine air-leak failures in them. Using data you already understand means the whole of today's attention goes on the model and the decision, not on learning a new domain.

Why this needs a safety case at all. Nobody is proposing the model applies the brake. But if a health monitor tells a depot "this unit is fine" and the depot believes it, the model has quietly become part of how the railway decides what to inspect. That is enough to require a written argument before it goes live.

2. What you will be able to do

- Explain what an autoencoder is and why squeezing data through a bottleneck teaches a network what "normal" looks like — without ever being shown a failure.
- Say why a neural network needs its inputs scaled when a random forest does not, and fit a scaler without leaking the test period into it.
- Turn reconstruction error into an operational health score, and set an alarm threshold from the training distribution alone.
- Score a rare-event detector with recall, precision and ROC-AUC — and translate all three into what the depot will actually experience.
- Measure how much of a deep model's performance is a property of the trained artefact rather than the design, by re-running it across random seeds.
- Explain why a training loss curve is not evidence that a model works.
- Distinguish what a model can be shown to do from what it is claimed to do — and design the experiment that would settle the difference.
- Write a structured safety case: claim, evidence for, evidence against, the cost of being wrong, residual risk, recommendation.

- Recognise when a favourable result comes from the data and when it comes from a choice the analyst made — a threshold grid, a smoothing convention, a random seed.

3. What's in your package

File	What it is
notebook/lab5_deep_learning_safety_case.ipynb	The notebook you work in — open this.
notebook/metropt3_teaching.csv	The real dataset (already provided). The same file as Lab 2. Keep it beside the notebook.
Safety_Case_Lab5.docx	Your written Safety Case — the main deliverable.
Activity_Dossier_Lab5.pdf	This document — your guide to the activity.
Safety_Case_Lab5.pdf	A read-only copy of the deliverable, if you prefer to read it on screen.
README_Lab5_data.md	Where the data comes from, and its terms of use.
START_HERE.txt	One-page quick start.

4. Before you start

This lab runs in Google Colab. You need a free Google account and nothing else — no installation, no setup.

Step	What you do
1	Go to colab.research.google.com and sign in.
2	File ► Upload notebook, and pick <code>lab5_deep_learning_safety_case.ipynb</code> . (Or use the one-click link your instructor shared, which skips this step.)
3	Runtime ► Change runtime type ► CPU. You do NOT need a GPU for this lab — picking one will not make anything faster.
4	Run the cells from the top, one at a time, with Shift+Enter.

The first cell fetches the dataset for you. If it cannot reach it, the cell says so and tells you to upload the CSV by hand — click the folder icon on the left, upload `metropt3_teaching.csv`, and re-run that one cell.

Two things about Colab that catch people out. Step 9 trains five neural networks and takes about a minute — it has not hung. And Colab disconnects idle sessions, losing every variable when it does. This lab has a stretch near the end, Challenge Part 2, with no code in it at all, where you are thinking rather than typing — which is exactly when a session tends to drop. If it happens: Runtime ► Run all, and you are back in about 90 seconds.

Running on a lab VM instead? The same notebook works unchanged — open a terminal in the notebook folder, run `jupyter lab`, and go. It picks up the local CSV automatically.

```
# On a lab VM, in the notebook folder:
jupyter lab
# then open lab5_deep_learning_safety_case.ipynb and run each cell top to bottom
```

How the notebook works: every step is fully worked and explained — read it, run it, then do the  **Your turn** cell yourself. There are nine of them, and the lab ends with a two-part  **Challenge**. Only one idea is new at a time.

One thing will feel odd, and it is deliberate. Most of this lab is spent finding reasons NOT to trust the model you have just built. That is not cynicism and it is not a trick question — it is the job. An engineer who can only make models is half an engineer; the other half is the one who can say, on the record and with numbers, what a model has not been shown to do.

5. The activity, step by step

Step	What you do	Already learned / New?
0-1	Load the data; check how rare failures are and whether the record is complete	Already learned (Labs 2 & 4)

Step	What you do	Already learned / New?
2	Why a neural network insists on scaled inputs — and fitting the scaler without leaking	NEW — StandardScaler
3	Split by time, then train on the NORMAL minutes only	NEW — unsupervised anomaly detection
4	Build and train the autoencoder; read the training loss curve	NEW — neural networks, epochs, bottleneck
5	Reconstruction error as a health score	NEW — the core idea of the product
6	Smooth over 15 minutes; set the threshold from training data only	Already learned (Lab 4's leakage rule)
7	Score it: recall, precision, ROC-AUC — then say it in depot language	Mostly known; ROC-AUC is new
8	The skeptic step: a control chart and a single raw sensor, judged on identical terms	Already learned (Lab 2 habit)
9	The seed lottery — retrain five times and watch the product change	NEW — the DL-specific failure mode
10	"Why did it alarm?" — and why a neural network struggles to say	Extends Lab 2's feature importances
	Challenge: what a false alarm costs (and what "a false alarm" even means), then testing the claim that sold the product	Your turn

6. What to hand in

Deliverable	Notes
Completed notebook	All steps run top to bottom; the nine  Your turn cells filled; the  Challenge attempted. In Colab: File ► Download ► Download .ipynb BEFORE you close the tab — nothing left in a Colab session survives.
Written Safety Case	Safety_Case_Lab5.docx, in your own words. Quote a number as evidence where it helps — but a document that is only pasted output scores no marks, and in the real world earns no signature.
Show & tell	Be ready to state your recommendation in one sentence, and name the single piece of evidence that most nearly changed your mind.

7. How you are assessed (100 marks)

Component	Marks	Full marks means
Worked steps run +  Your turn cells done	25	Notebook runs top to bottom; all nine practice cells completed with a written interpretation, not just code
 Challenge (both parts)	20	Notifies that the threshold grid and the cost unit each change the answer; Part 2 names leave-one-fault-type-out validation and states that this dataset cannot run it
A claim narrow enough to be defensible	15	Scope is stated explicitly — one unit, one fault type, one held-out episode — rather than "the monitor detects APU

Component	Marks	Full marks means
		failures"
Evidence against, stated as plainly as evidence for	25	Covers the seed lottery with numbers, the no-ML baselines out-detecting the model, and the untested generalisation claim — with any cost figure quoted in a stated unit. A case that omits any of the three cannot score above half here.
Residual risk and a signed recommendation	15	Names who carries the risk, states deploy/trial/decline, and says what evidence would change the answer

8. Your data — where it comes from

The dataset is real: sensor readings from the Air Production Unit (the compressor that feeds the brakes and air suspension) of a metro train operated by Metro do Porto, Portugal, recorded minute by minute between February and June 2020. It is published as the MetroPT-3 dataset by the UCI Machine Learning Repository (dataset 791), under a Creative Commons Attribution 4.0 licence — free to use, including commercially, with attribution.

This is the same file you used in Lab 2, unchanged. Two real air-leak failures are marked in it: one on 18 April 2020, one on 29–30 May 2020. The lab trains on the period containing the first and is graded on the second.

Column	Meaning
timestamp	The minute the readings were taken. Note: the record is NOT continuous — see Step 1.
15 sensor columns	TP2, TP3, H1, DV_pressure, Reservoirs, Oil_temperature, Motor_current, COMP, DV_electric, Towers, MPG, LPS, Pressure_switch, Oil_level, Caudal_impulses — pressures, temperatures, currents and valve states
airleak_failure	1 during a real air-leak failure, 0 otherwise. The network is never shown this column.

The number that governs everything you will write: this dataset contains exactly TWO failure episodes, on ONE unit, of ONE fault type. Whatever score you measure today is a measurement with a sample size of one held-out event. No modelling choice makes that number bigger.

9. Getting unstuck

- Run cells from the top, in order — later cells depend on variables created earlier.
- If the first cell cannot find the dataset, it tells you what to do: click the folder icon on the left in Colab, upload metropt3_teaching.csv, and re-run that cell.
- If Colab disconnects, Runtime ► Run all puts everything back in about 90 seconds. Nothing is lost except time.
- Step 9 trains five networks in a loop and takes about a minute. It has not hung.
- Your numbers should match the ones quoted in the notebook exactly, because random_state is fixed. If they do not, check you have not changed a seed or re-run a cell out of order.
- Never fit the scaler, or choose a threshold, using the test period. If a score suddenly improves, that is the first thing to check.
- A chart without a title and axis labels is not finished.

Stuck? Re-read the step's Insight box, then ask a teaching assistant.